

Clase 5

React

IIC2513 - Tecnologías y Aplicaciones Web

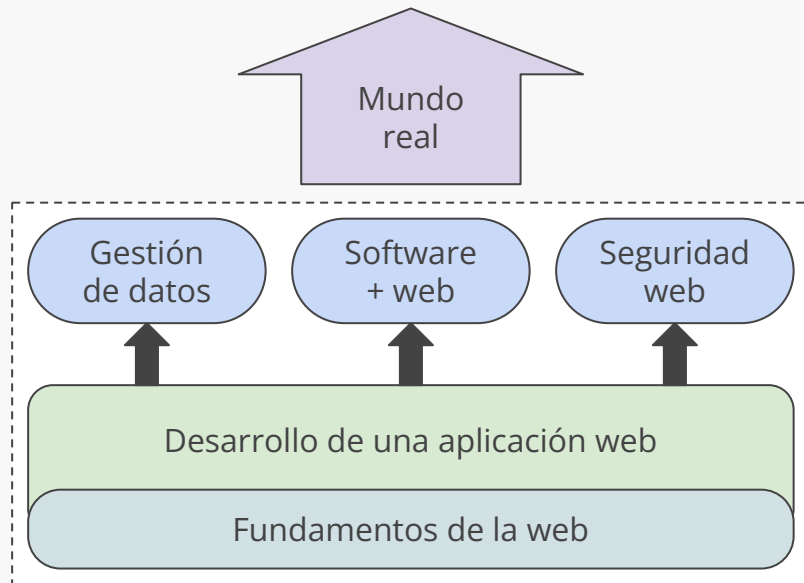
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Recuerden responder la ECA
2. Recuerden hacer el Control 1

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

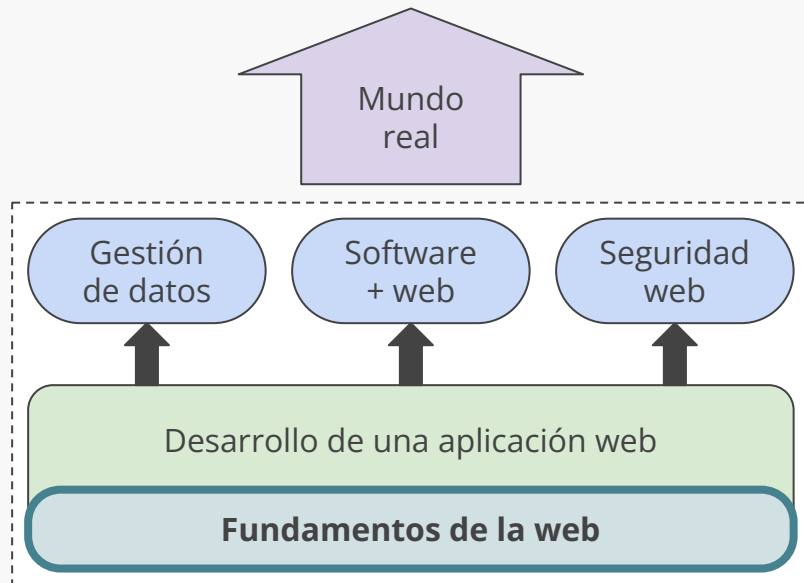


Contenidos del curso

Fundamentos de la web

Veremos definiciones relacionadas con internet, protocolos y qué son **HTML**, **CSS** y **JavaScript**...

... para comprender la web como una **plataforma de desarrollo**, su funcionamiento básico, y protocolos y tecnologías detrás de esta

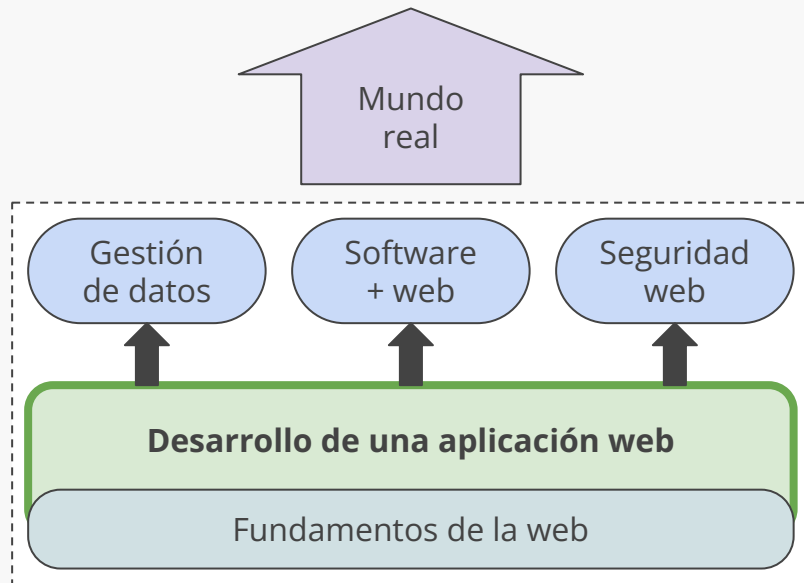


Contenidos del curso

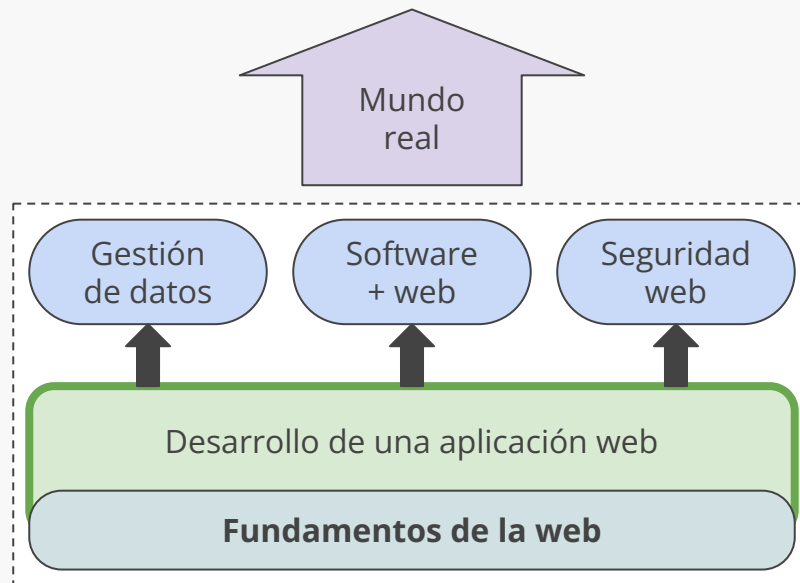
Desarrollo de una aplicación web

Veremos herramientas como **React**, Node.js, Koa y Express, y técnicas como CSS avanzado y el uso de *Design Systems*

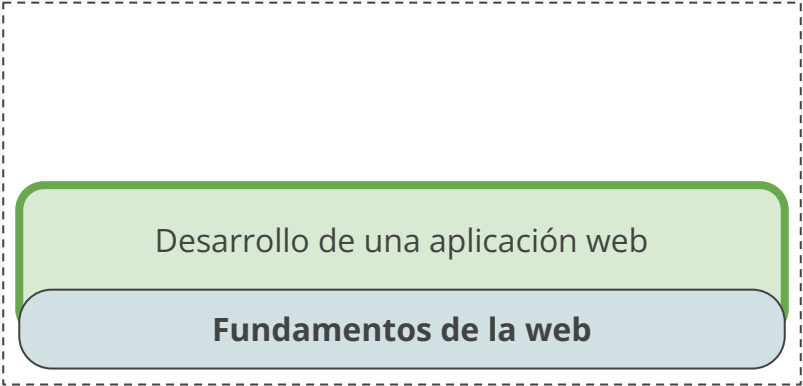
... para desarrollar **aplicaciones modernas** tanto en el lado del **cliente** (front-end) como en el lado del servidor (back-end)



Contenidos del curso



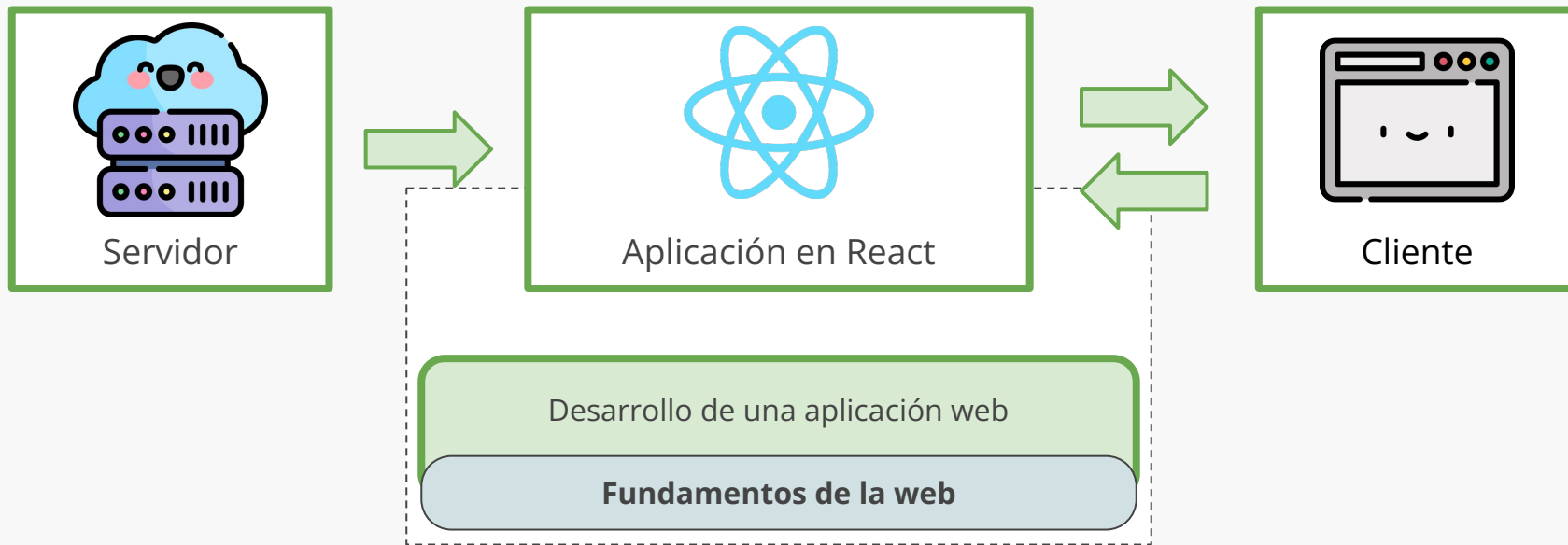
Contenidos del curso



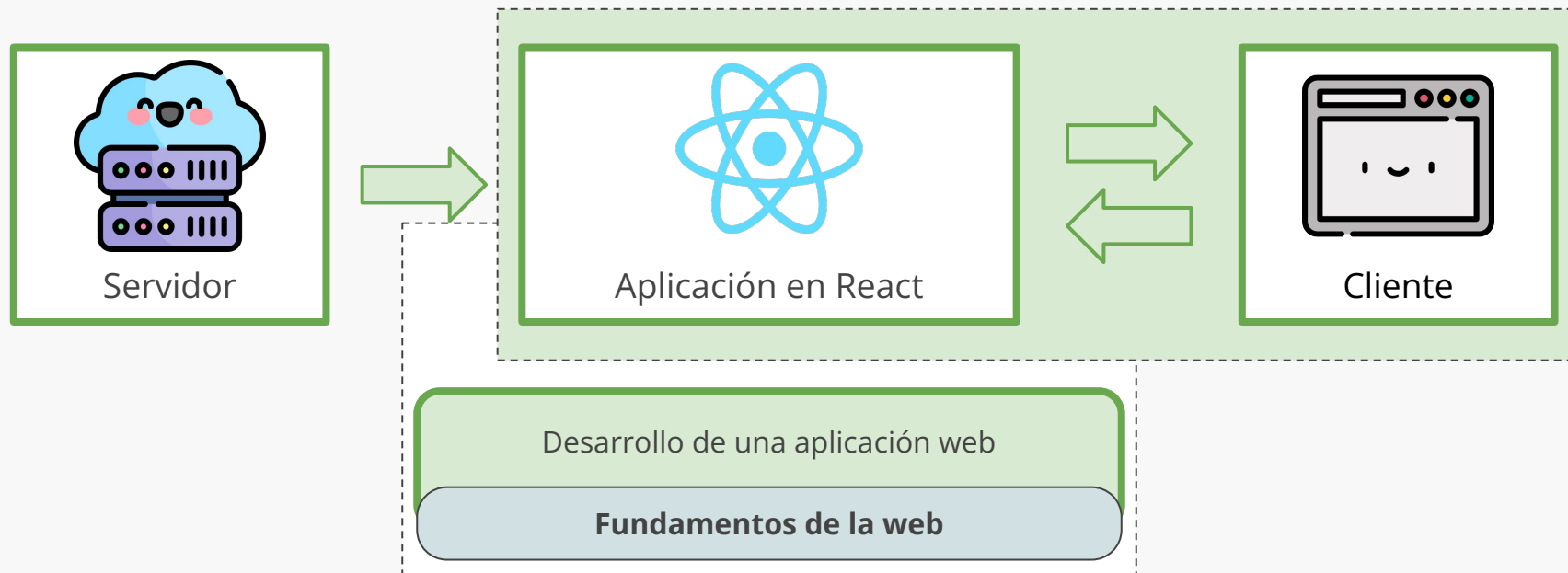
Desarrollo de una aplicación web

Fundamentos de la web

Contenidos del curso



Contenidos del curso

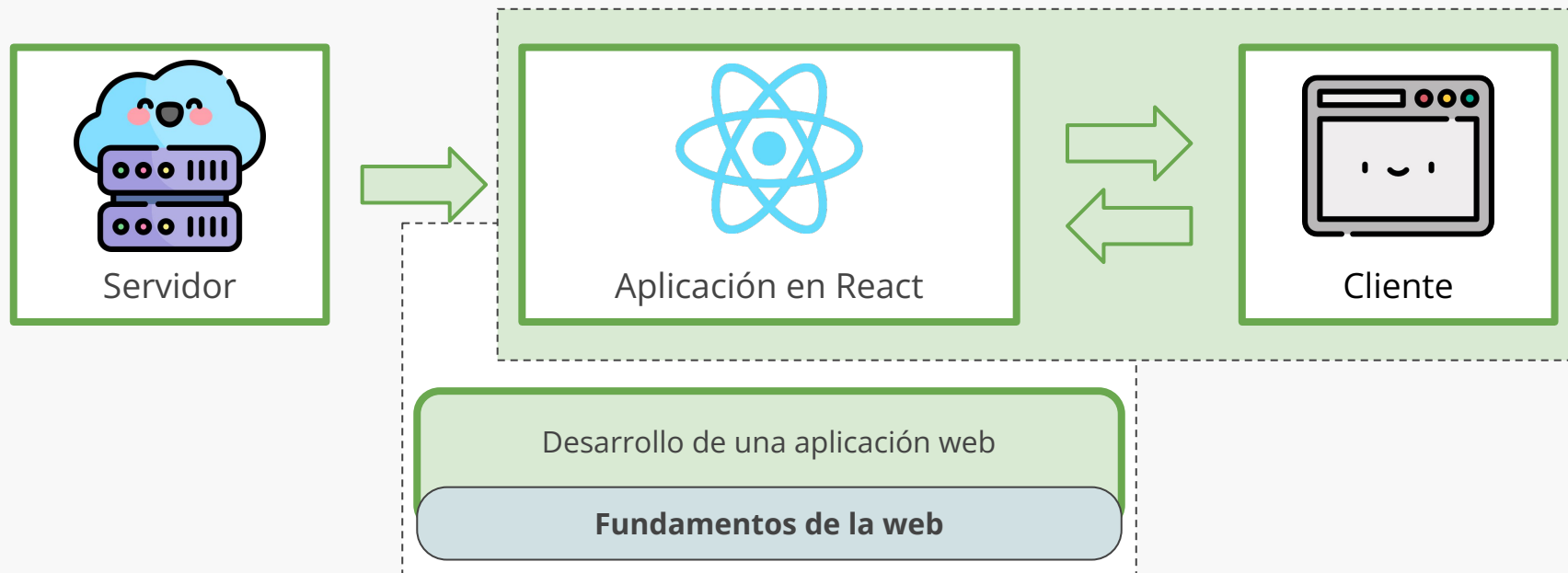


Contenidos del curso

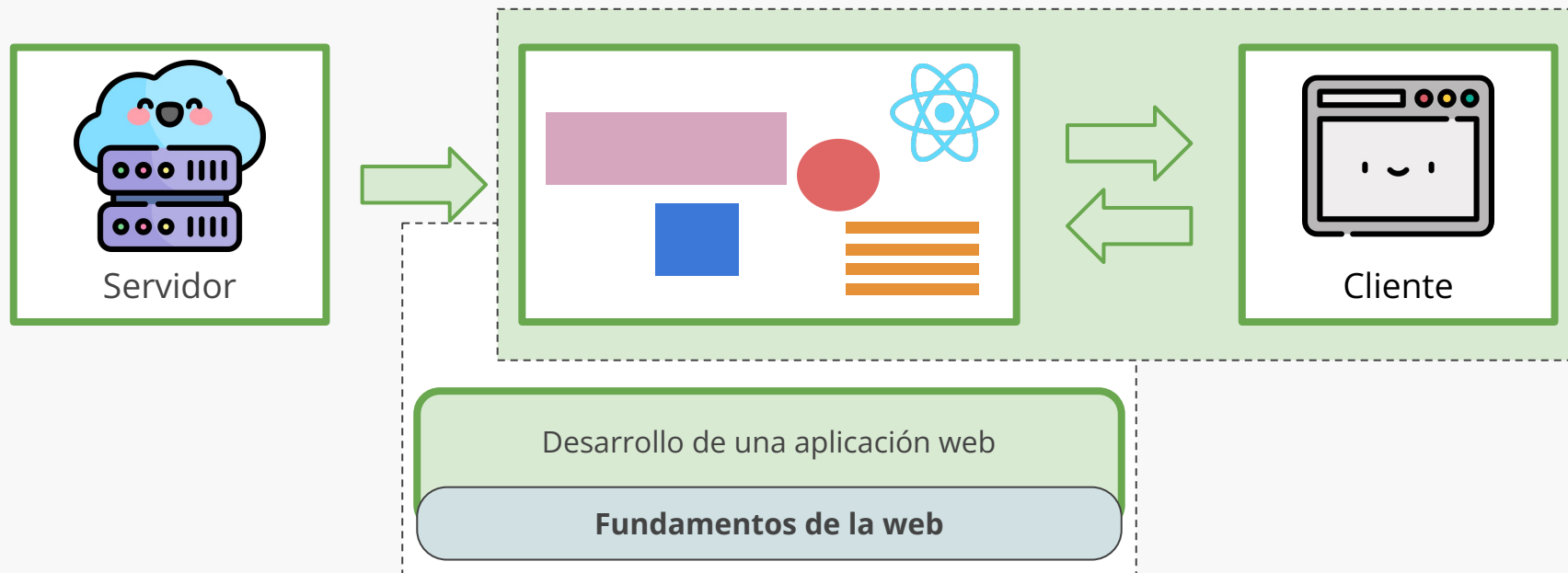
Pregunta# ¿Qué es React? ¿Cómo funciona? ¿Cómo construimos aplicaciones web con esto?



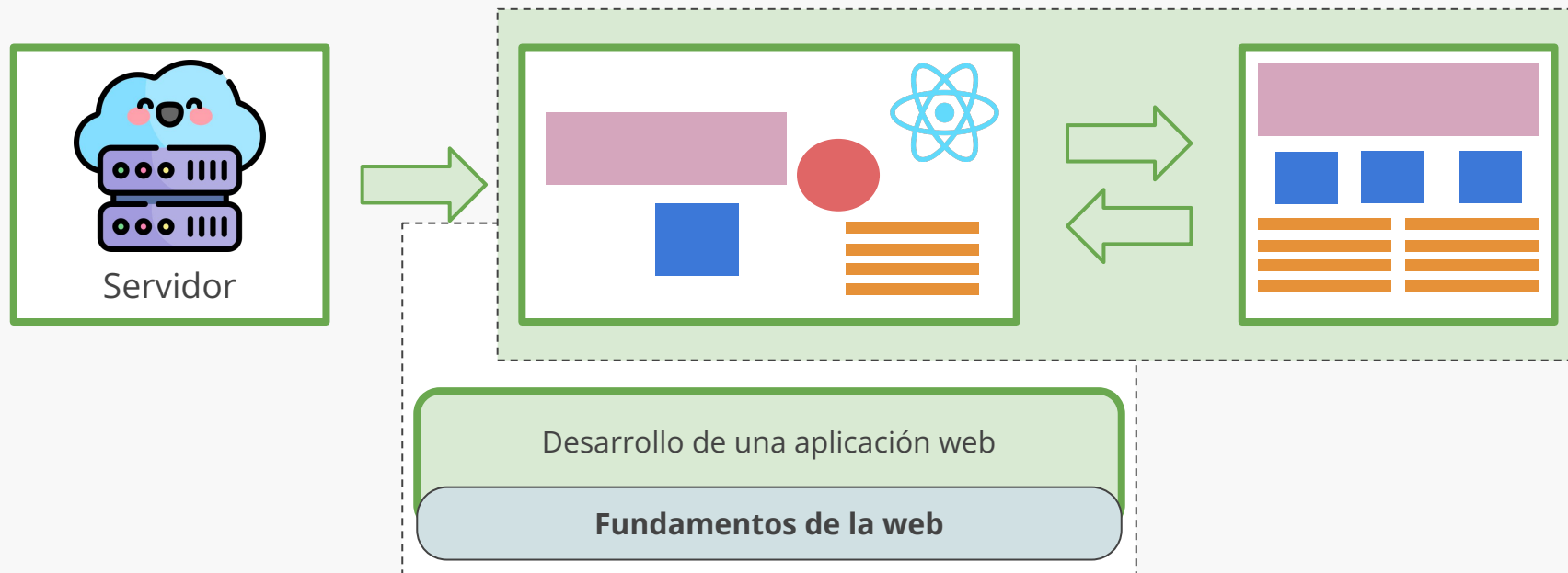
Contenidos del curso



Contenidos del curso



Contenidos del curso

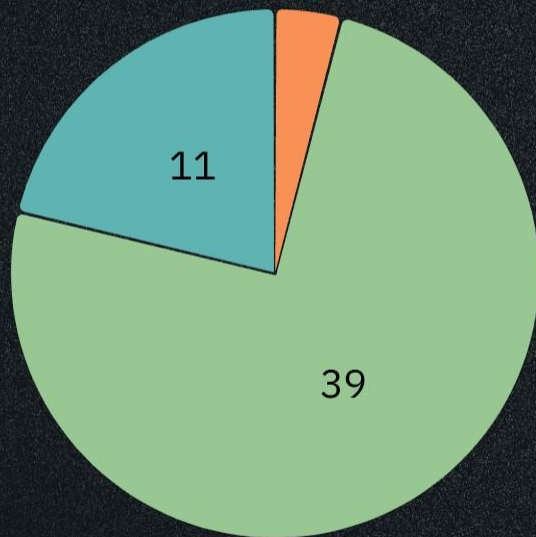


Menti#*Unas preguntas antes de empezar*

El código en las diapositivas debería...

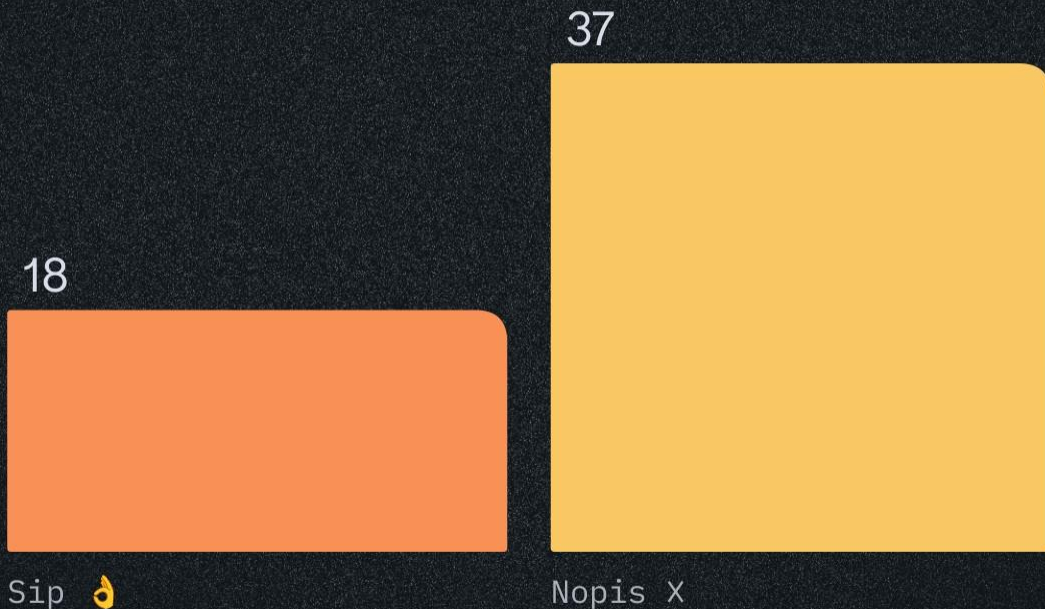


¿Cómo te fue con la Actividad 2? 🍕

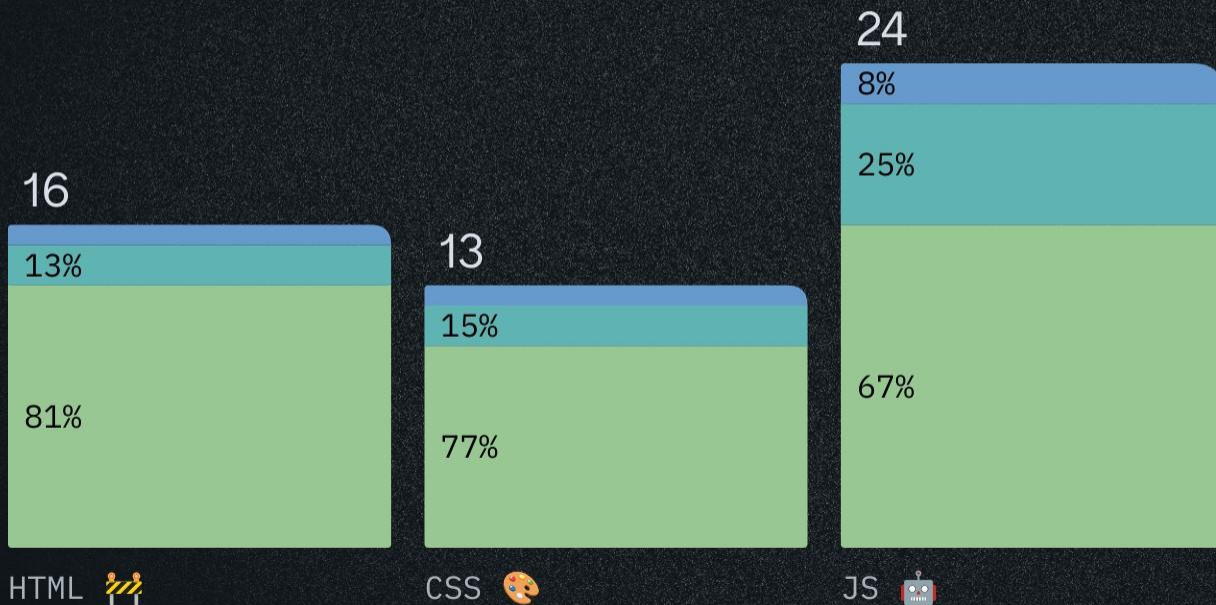


- 2 No la hice 💀
- 0 Algo hice 👉👉
- 39 Pude completarla 💯
- 11 La encontré fácil 😎

¿Fueron a la ayudantía de JavaScript? 🧐



¿Has practicado esto por tu cuenta este mes? 🤔



Segment

¿Cómo te fue con la Actividad 2? 🍕

- No la hice 💀
- Algo hice 🙌🙌
- Pude completarla 100
- La encontré fácil 😎
- Unknown

¡Gracias por responder!

Agenda

🌟 React

Importante:

Esta clase es una **introducción a React**

Hay muchos elementos adicionales de esta librería que están bien documentados, algunos que veremos en **próximas clases** 🙌 , y otros que tendrán que **aprender por su cuenta** 📺📚

Agenda

🌟 React

Conceptos importantes

Pensando en componentes

Manejo de eventos

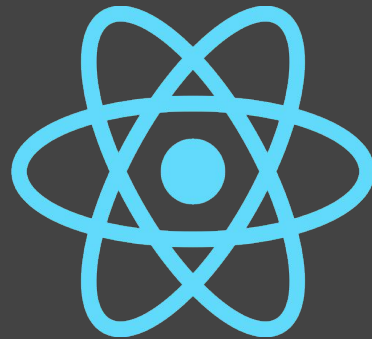
Un proyecto React

React

Librería de Javascript (*open source*) para crear **interfaces de usuario (UI)** en aplicaciones dinámicas

Proyecto iniciado por Facebook (primera versión lanzada en 2013) y mantenido por la misma empresa y la comunidad *open source*. Actualmente en su versión 19 (Dic. 2024)

react.dev

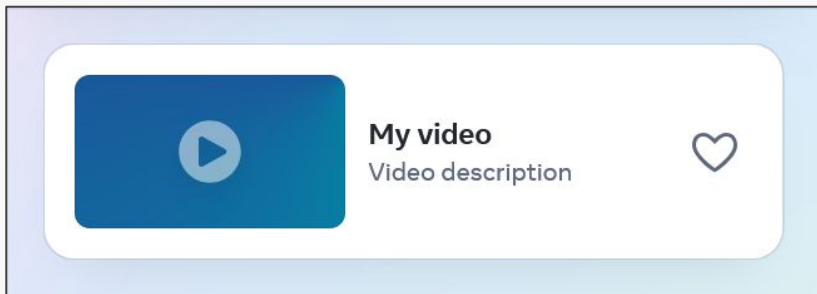


```
function Video({ video }) {  
  return (  
    <div>  
      <Thumbnail video={video} />  
      <a href={video.url}>  
        <h3>{video.title}</h3>  
        <p>{video.description}</p>  
      </a>  
      <LikeButton video={video} />  
    </div>  
  );  
}
```

React

React es la **vista** en un contexto en el que se use el **patrón MVC** y se basa en el concepto de “componente”

¿Framework o librería? React solo **especifica el proceso de rendering** y todo lo demás queda a elección del proyecto, por lo que no es un framework



```
function Video({ video }) {  
  return (  
    <div>  
      <Thumbnail video={video} />  
      <a href={video.url}>  
        <h3>{video.title}</h3>  
        <p>{video.description}</p>  
      </a>  
      <LikeButton video={video} />  
    </div>  
  );  
}
```




v19

🔍 Search

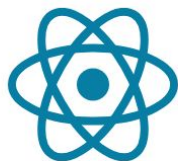
Ctrl K

Learn

Reference

Community

Blog



React

The library for web and native user interfaces

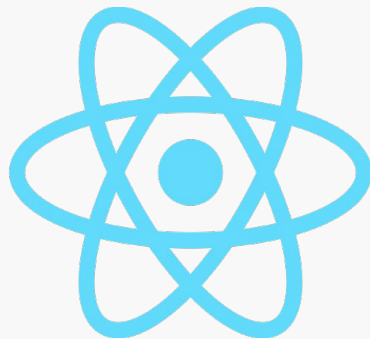
Learn React

API Reference

¿Cuándo ocupamos React?

React es muy útil en aplicaciones donde hay múltiples cambios de estado que se relacionan o dependen de otros, y donde queremos generar contenido de manera dinámica:

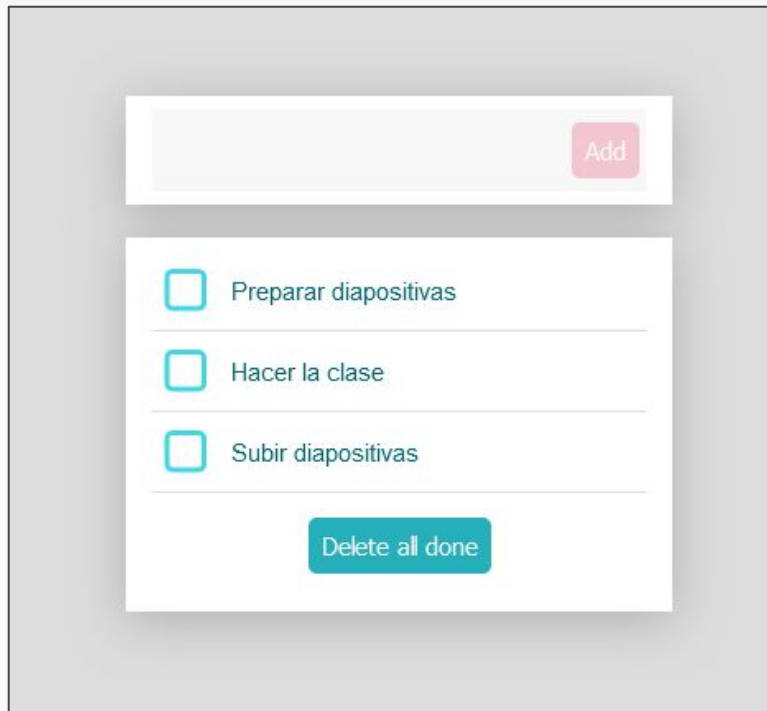
- *Dashboards* o herramientas de visualización
- UIs basadas en componentes
- Cuando se requiere actualizar el DOM rápidamente
- Cuando queremos reutilizar componentes entre app web y móvil
- ...
- **Y en el proyecto del curso**



Arquitectura basada en componentes

Componente

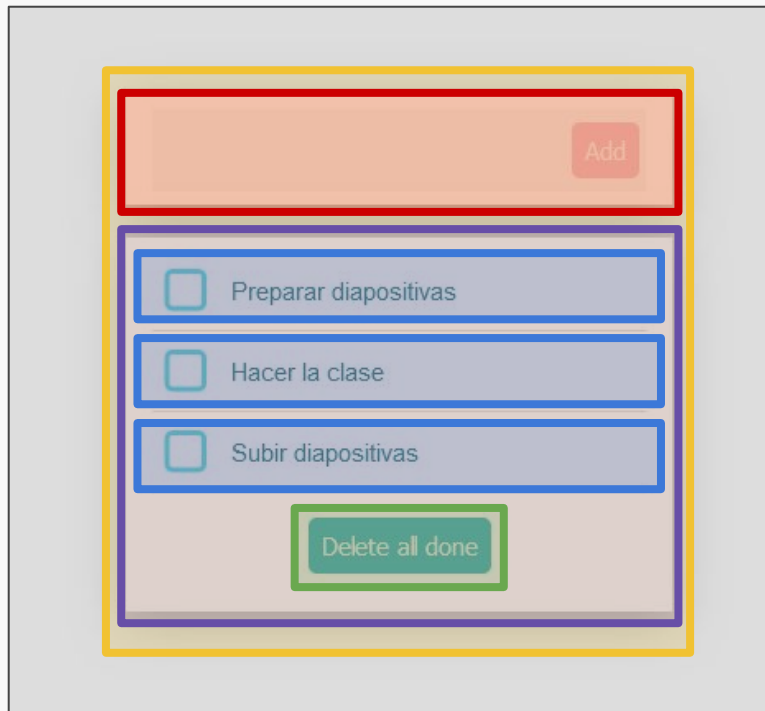
- Trozo de código que describe una porción de UI
- Reusable y fácil de mantener
- Componentes se pueden componen entre ellos



Arquitectura basada en componentes

Componente

- Trozo de código que describe una porción de UI
- Reusable y fácil de mantener
- Componentes se pueden componen entre ellos



State of JS: Encuesta anual a desarrolladores sobre el ecosistema JavaScript

Other Surveys:

[State of AI](#)

[State of CSS](#)

[State of Devs](#)

[State of GraphQL](#)

[State of HTML](#)

[State of React](#)

State of JavaScript

The annual developer survey of the JavaScript ecosystem

MOST RECENT

2024

STATE OF

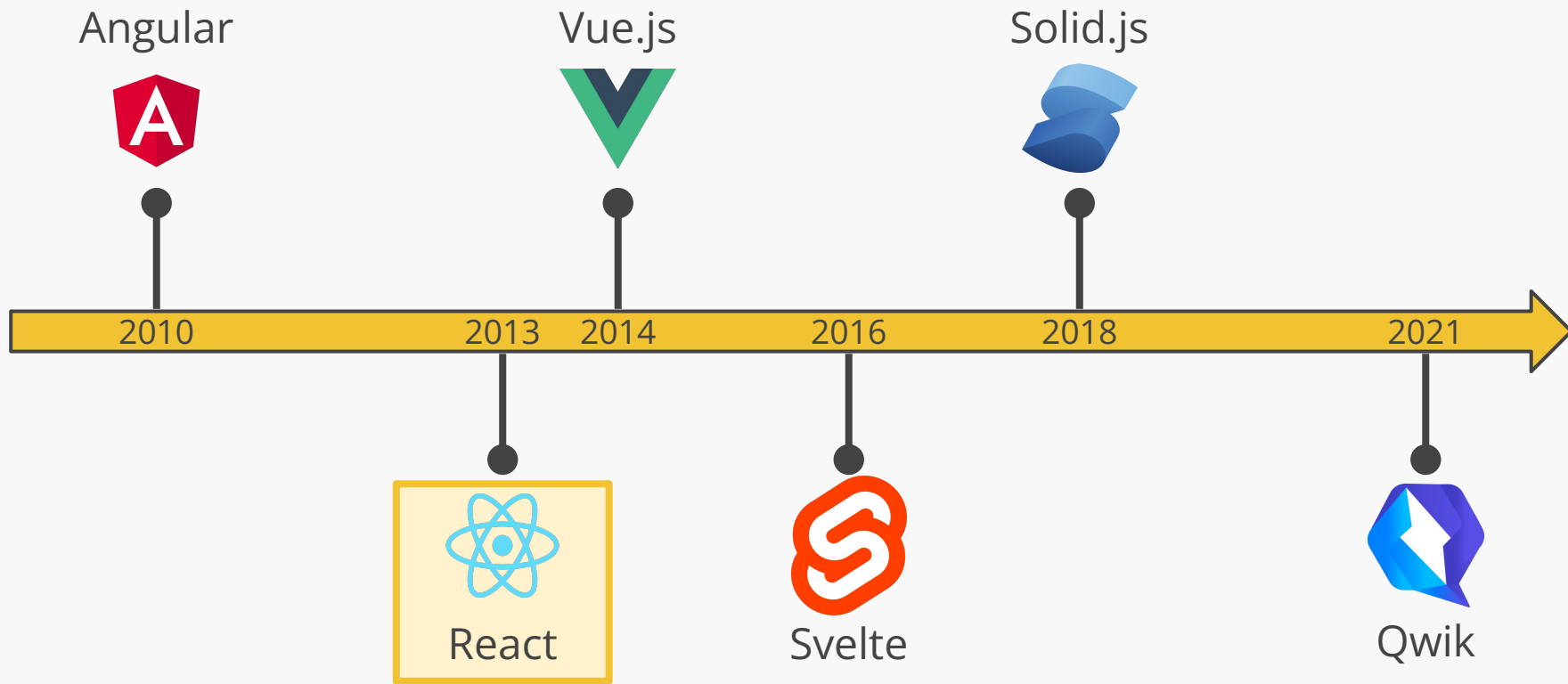


2024

[View Questions](#)

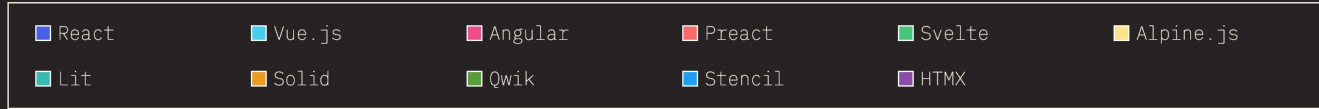
[View Results](#)

Línea de tiempo de frameworks populares



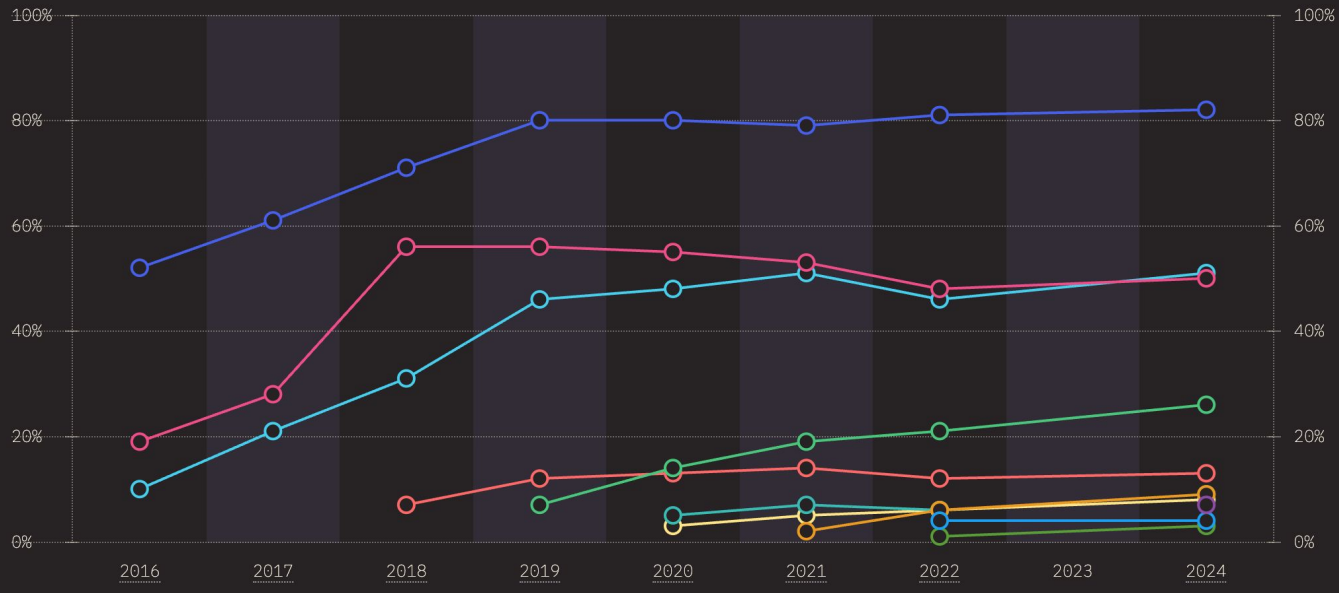
Fuente: [State of JavaScript 2022: Front-end Frameworks | Front-end frameworks and libraries](#)

FRONT-END FRAMEWORKS RATIOS OVER TIME



Mode: **Value** Rank

View: **Usage** Awareness Interest Retention Positivity



FRONT-END FRAMEWORKS EXPERIENCE & SENTIMENT

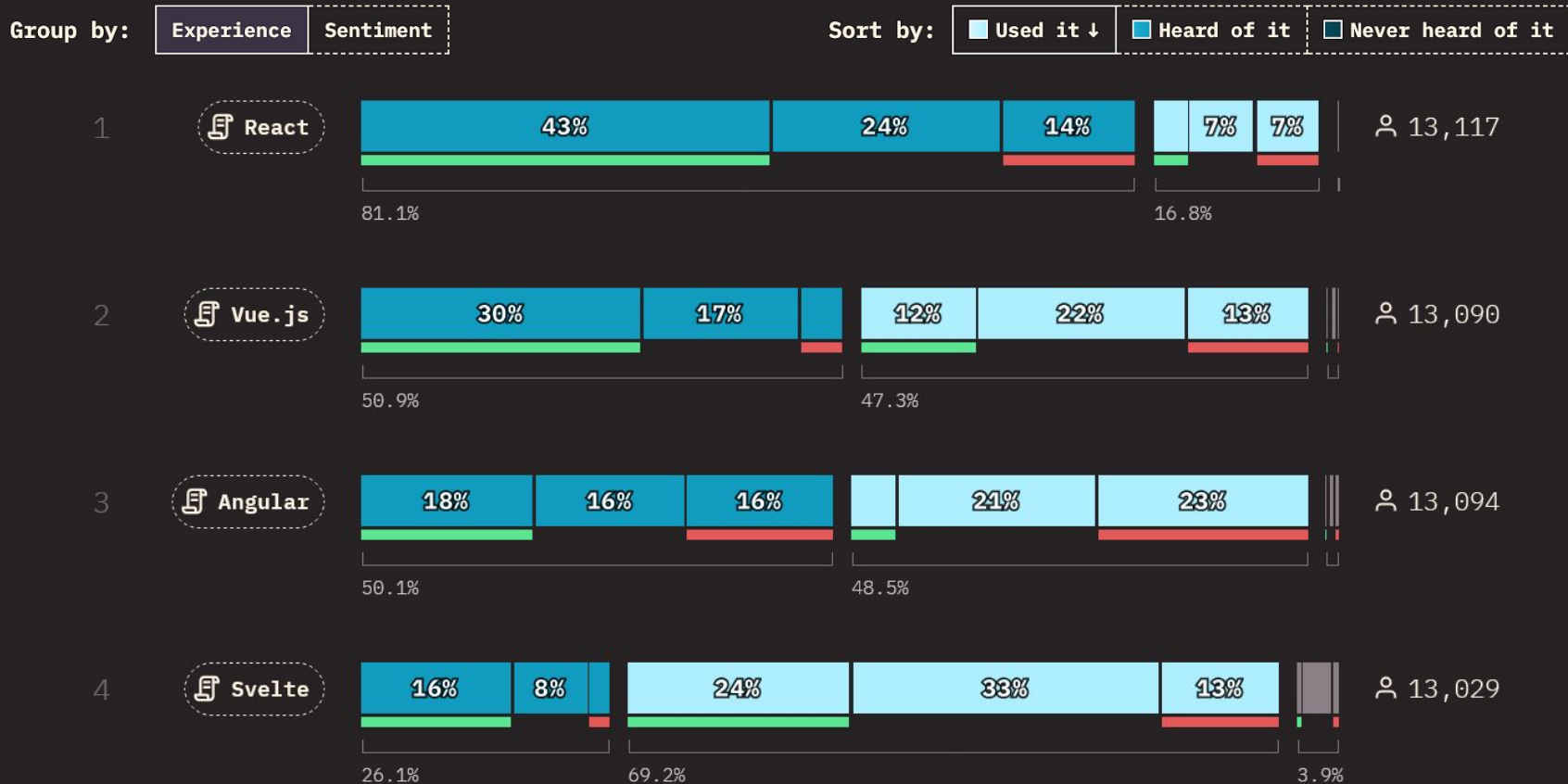


Figura: [State of JavaScript 2023: Front-end Frameworks](#)

Agenda

✨ React

✨ Conceptos importantes

Pensando en componentes

Manejo de eventos

Un proyecto React

Rendering

Una aplicación React se renderiza sobre un elemento existente en el DOM:

```
import React from 'react';  
import ReactDOM from 'react-dom/client';  
  
import { App } from './App.jsx';  
  
const rootNode = document.getElementById('#root');  
  
ReactDOM.createRoot(rootNode).render(<App />);
```

Elemento

Un elemento es la unidad mínima de una aplicación React. Se traduce en elementos del DOM directamente, por lo que podemos componer *tags* HTML u otros componentes:

Reference

`createElement(type, props, ...children)`

Call `createElement` to create a React element with the given `type`, `props`, and `children`.

```
import { createElement } from 'react';

function Greeting({ name }) {
  return createElement(
    'h1',
    { className: 'greeting' },
    'Hello'
  );
}
```

Figura: [createElement – React](#)

Elemento

Un elemento es la unidad mínima de una aplicación React. Se traduce en elementos del DOM directamente, por lo que podemos componer *tags* HTML u otros componentes:

```
const element = React.createElement("h1", null, "Un título");
```

- **type**: nombre de *tag* o componente de React
- **props**: el elemento será creado con estos *props* (elementos en HTML, *properties* en JS)
- (optional) **...children**: cero o más nodos hijos (nodos/elementos React, *strings*, números, nodos, etc.)

<h1>Un título</h1>

Elemento

Un elemento es la unidad mínima de una aplicación React. Se traduce en elementos del DOM directamente, por lo que podemos componer *tags* HTML u otros componentes:

```
import { createElement } from 'react';

function Greeting({ name }) {
  return createElement(
    'h1',
    { className: 'greeting' },
    'Hello ',
    createElement('i', null, name),
    '. Welcome!'
  );
}
```

```
<div id="root">
  <h1 class="greeting">Hello <i>Taylor</i>. Welcome!</h1>
</div>
```

Elemento

Un elemento es la unidad mínima de una aplicación React. Se traduce en elementos del DOM directamente, por lo que podemos componer *tags* HTML u otros componentes:

```
import { createElement } from 'react';

function Greeting({ name }) {
  return createElement(
    'h1',
    { className: 'greeting' },
    'Hello ',
    createElement('i', null, name),
    '. Welcome!'
  );
}
```

```
function Greeting({ name }) {
  return (
    <h1 className='greeting'>
      Hello <i>{name}</i>. Welcome!
    </h1>
  );
}
```

```
<div id="root">
  <h1 class="greeting">Hello <i>Taylor</i>. Welcome!</h1>
</div>
```

Elemento

Un elemento es la unidad mínima de una aplicación React. Se traduce en elementos del DOM directamente, por lo que podemos componer *tags* HTML u otros componentes:

```
import { createElement } from 'react';

function Greeting({ name }) {
  return createElement(
    'h1',
    { className: 'greeting' },
    'Hello ',
    createElement('i', null, name),
    '. Welcome!'
  );
}
```

¿JavaScript o HTML?

```
function Greeting({ name }) {
  return (
    <h1 className='greeting'>
      Hello <i>{name}</i>. Welcome!
    </h1>
  );
}
```

```
<div id="root">
  <h1 class="greeting">Hello <i>Taylor</i>. Welcome!</h1>
</div>
```

Elemento

Un elemento es la unidad mínima de una aplicación React. Se traduce en elementos del DOM directamente, por lo que podemos componer *tags* HTML u otros componentes:

```
import { createElement } from 'react';

function Greeting({ name }) {
  return createElement(
    'h1',
    { className: 'greeting' },
    'Hello ',
    createElement('i', null, name),
    '. Welcome!'
  );
}
```

Ninguno y ambos: es JSX

```
function Greeting({ name }) {
  return (
    <h1 className='greeting'>
      Hello <i>{name}</i>. Welcome!
    </h1>
  );
}
```

```
<div id="root">
  <h1 class="greeting">Hello <i>Taylor</i>. Welcome!</h1>
</div>
```


JSX: JavaScript + XML

JSX es una extensión a la sintaxis de JavaScript. Es similar a un lenguaje de templado, pero incluye todo lo que puede hacer JavaScript. Al igual que React, también fue desarrollado por Facebook. En React, lo podemos ocupar para describir cómo esperamos que una parte de la interfaz se vea:

```
// Un string en JS? Es HTML? No, es JSX  
const element = <h1>Hello, world!</h1>;
```

React no requiere JSX pero, aunque es opcional, es muy útil y más amigable que solo usar objetos de React directamente

JSX: JavaScript + XML

```
// Un string en JS? Es HTML? No, es JSX
const element = <h1>Hello, world!</h1>;

const user = 'Antonio';
// Puedo colocar cualquier {expresion} de JS
const element = <h1>Hello, {user}!</h1>;
const desc = <h3>My favourite {isHuman ? 'human' : 'thing'}</h3>;

const anchor = <a href='www.reactjs.org'>link</a>;
const img = <img src={user.avatarUrl}></img>;
```

JSX: JavaScript + XML

```
const profile = (  
  <div>  
    <h1>{user.username}</h1>  
    <h2>Seen movies</h2>  
    <ul>  
      <li>{movies[0]}</li>  
      <li>{movies[1]}</li>  
      <li>{movies[2]}</li>  
    </ul>  
    <ul>  
      {movies.map((m, i) => (  
        <li>{m}</li>  
      ))}  
    </ul>  
  </div>  
);
```

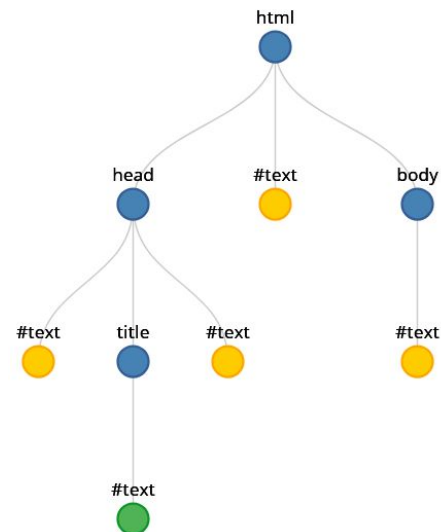
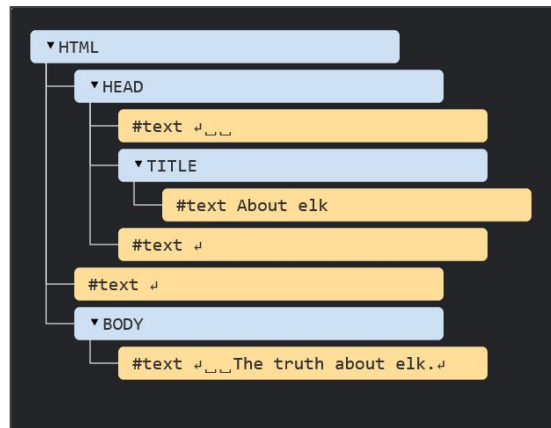
¿Por qué conviene usar JSX?

```
// Sintaxis "regular"  
function App() {  
  return createElement(Greeting, { name: 'Taylor' });  
}  
  
// Sintaxis con JSX  
function App() {  
  return <Greeting name='Taylor' />;  
}
```

HTML DOM

La representación interna de un documento HTML se llama **DOM (document object model)**. El DOM es un modelo jerárquico, basado en un árbol, donde *tags* y todo elemento en el HTML son representados como nodos, construyéndose según la jerarquía de la página desde el *tag* html. Estandarizado por la W3C ([ver estándar](#))

```
<!DOCTYPE HTML>
<html>
<head>
  <title>About elk</title>
</head>
<body>
  The truth about elk.
</body>
</html>
```



HTML DOM desde JavaScript

El HTML DOM es un modelo de objeto estandarizado y una interfaz de programación para HTML. Define:

- Los elementos HTML como **objetos**
- Las **propiedades** de todos los elementos HTML
- Los **métodos** para acceder a todos los elementos HTML
- Los **eventos** para todos los elementos HTML

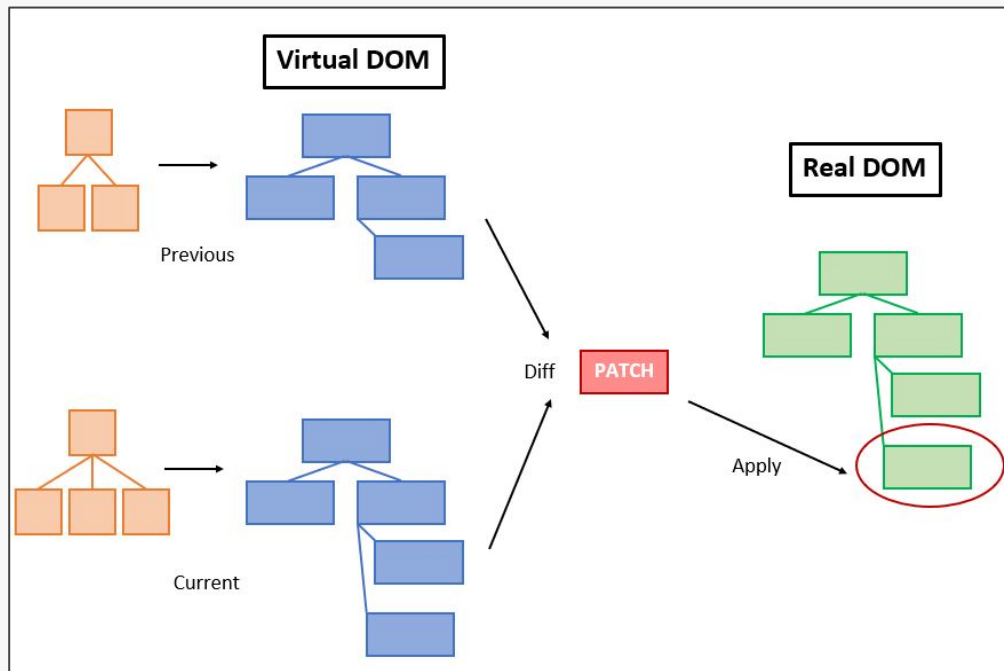
Esto convierte al HTML DOM es un estándar sobre cómo obtener, cambiar, agregar o eliminar elementos HTML:

- **JavaScript** puede hacer consultas y cambios sobre el DOM
- Accesible como **variable global** en el navegador (window, this)

Virtual DOM

Virtual DOM (VDOM) es una representación ideal “virtual” de la UI en memoria que se mantiene en **sincronía con el DOM “real”** (en React, esto lo hace la librería ReactDOM)

Cambios de UI en un componente gatillan una **comparación entre Virtual DOM y DOM real**. Se utiliza un *diff* para actualizar DOM de forma rápida



Virtual DOM

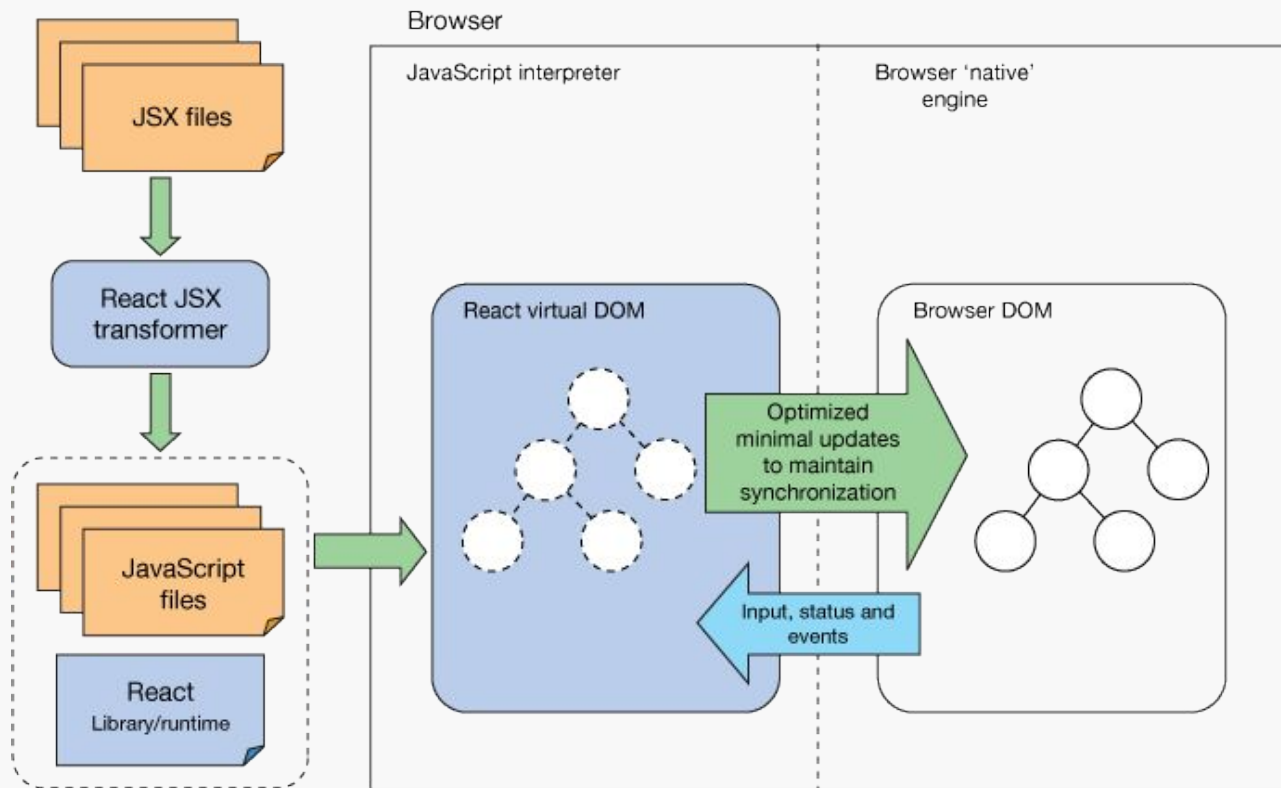


Figura: [React: Create maintainable, high-performance UI components](#) | IBM Developer

Agenda

✨ React

Conceptos importantes

✨ Pensando en componentes

Manejo de eventos

Un proyecto React

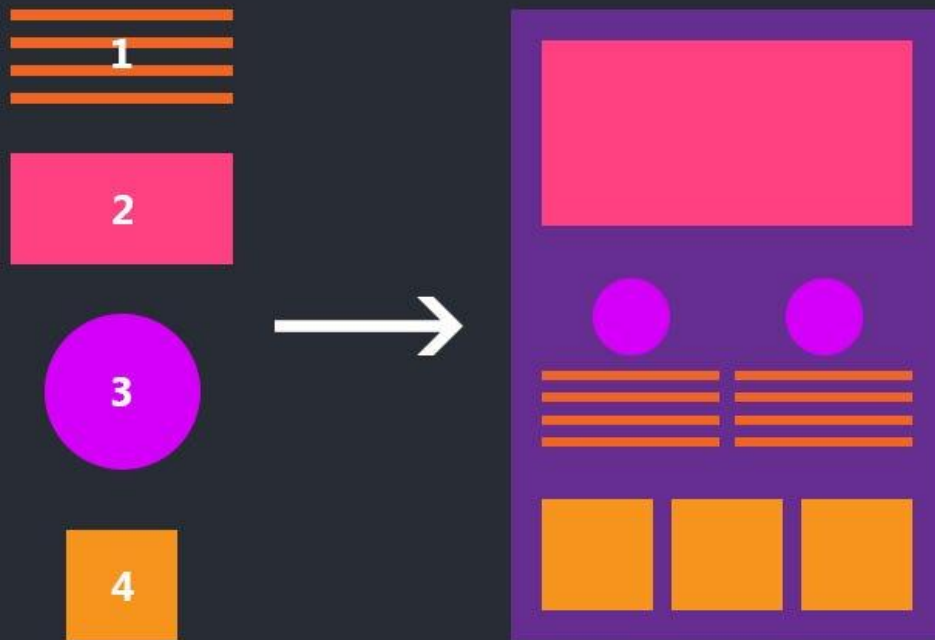
Componentes

React permite describir elementos de UI **reusables y combinables**

Son trozos de código que describen una porción de la UI:

- Recibe input y retorna lo que debe aparecer visualmente
- Compuesto de elementos React

React Components



Tipos de componentes

Sintaxis de función de JavaScript. Forma recomendada de escribir componentes

```
// Opción 1: componente funcional
function MyComponent(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

Pitfall

We recommend defining components as functions instead of classes. [See how to migrate.](#)

Sintaxis de clase de JavaScript (ES6). Útil cuando sí necesitamos pasar información de un componente a otro

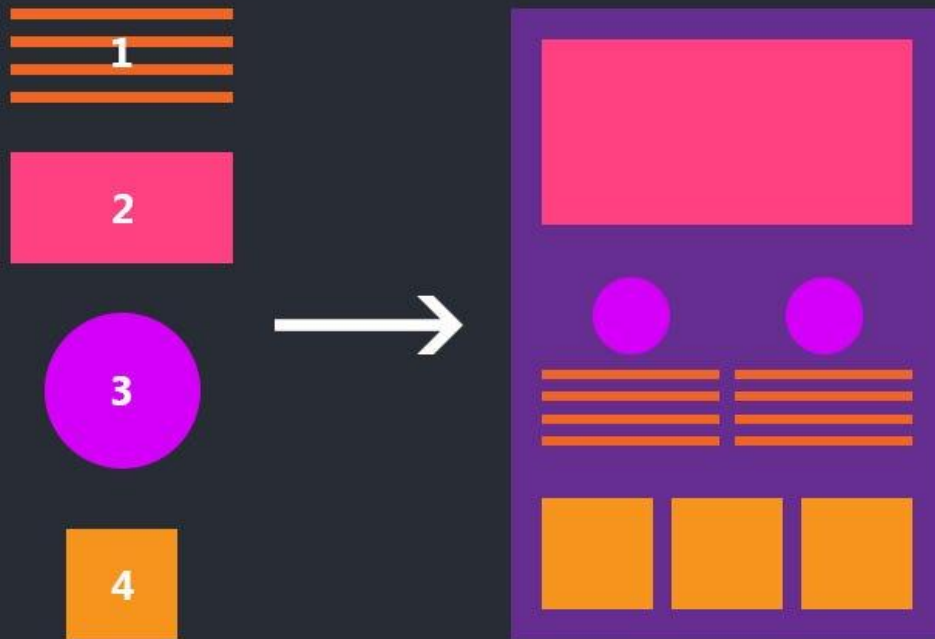
```
// Opción 2: componente de clase
class Counter extends React.Component {
  constructor(props) {
    super(props);
  }

  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Componentes

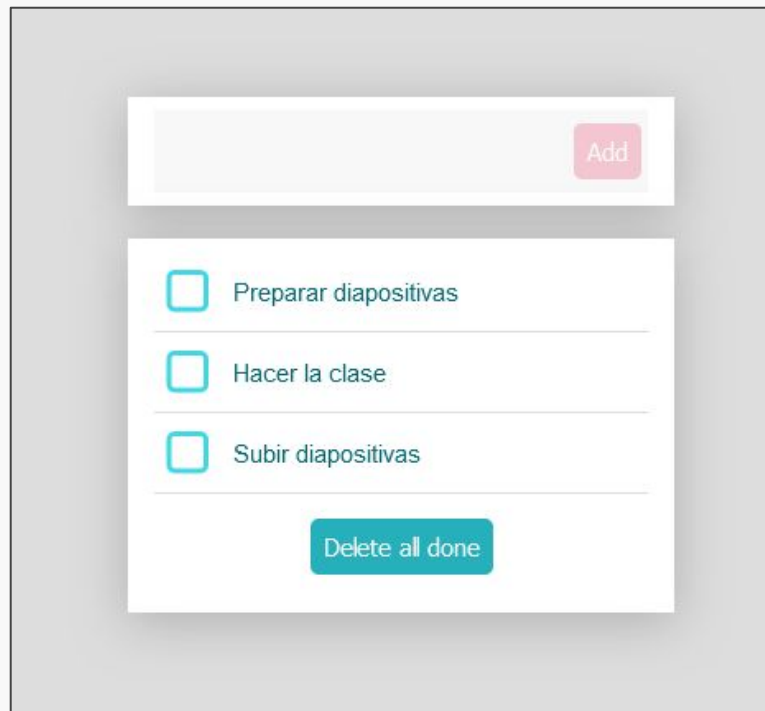
```
function Example() {  
  return (  
    <PageLayout>  
      <Header />  
      <div>  
        <AuthorBio name='A' />  
        <AuthorBio name='B' />  
      </div>  
      <div>  
        <BusinessImg com='A' />  
        <BusinessImg com='B' />  
        <BusinessImg com='C' />  
      </div>  
    </PageLayout>  
  );  
}
```

React Components



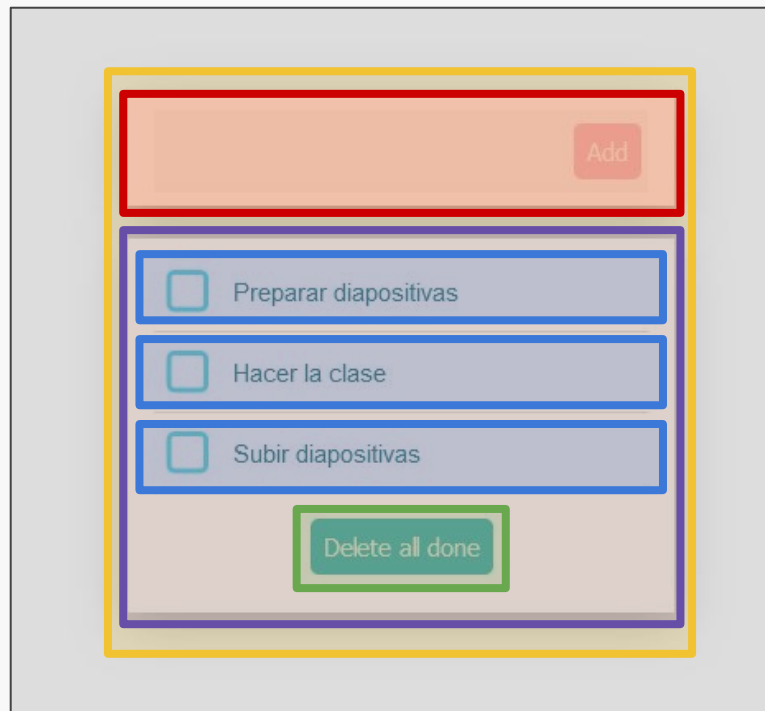
¿Cómo comenzar?

1. Identificar y nombrar componentes
2. ...
3. ...



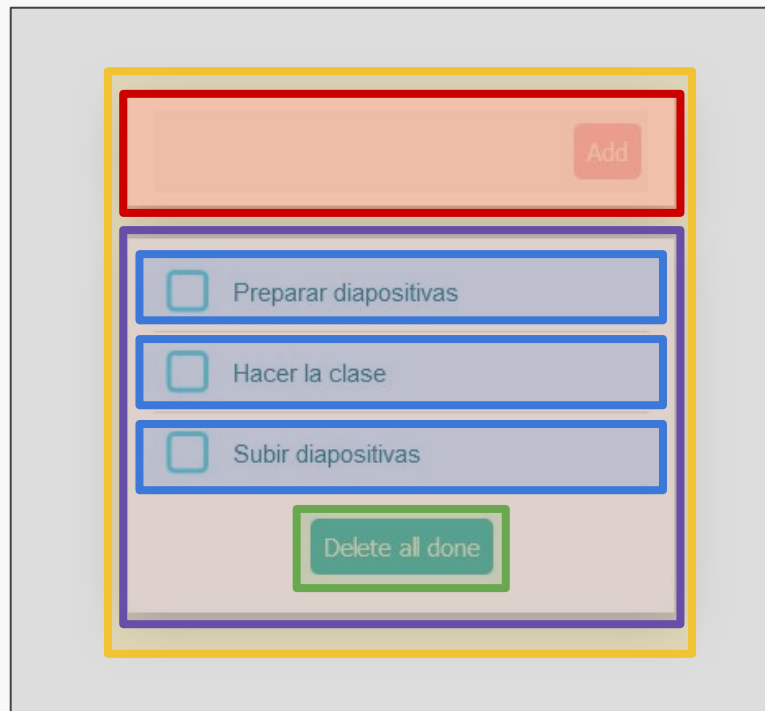
¿Cómo comenzar?

1. Identificar y nombrar componentes
2. ...
3. ...

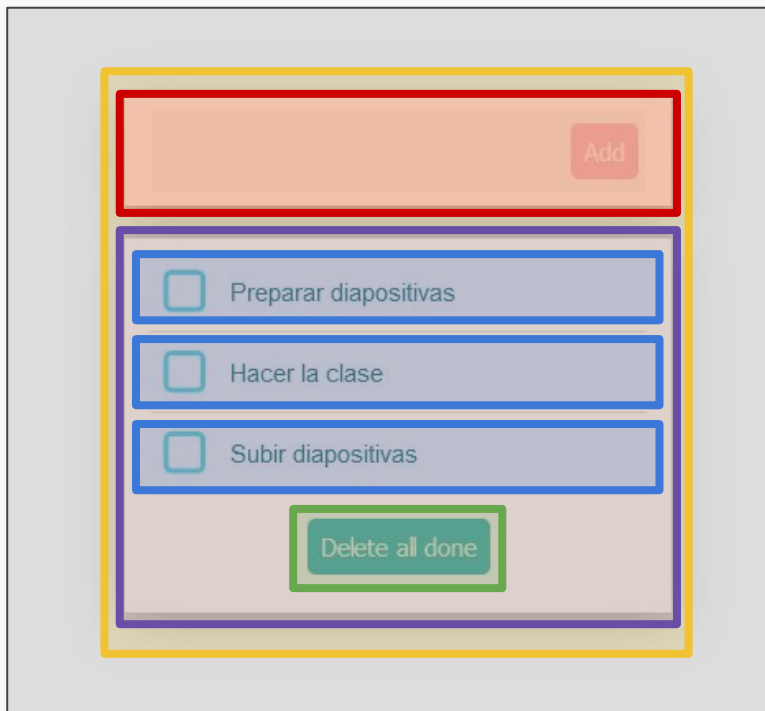


¿Cómo comenzar?

1. Identificar y nombrar componentes
2. Construir una versión estática
3. ...



¿Cómo comenzar?

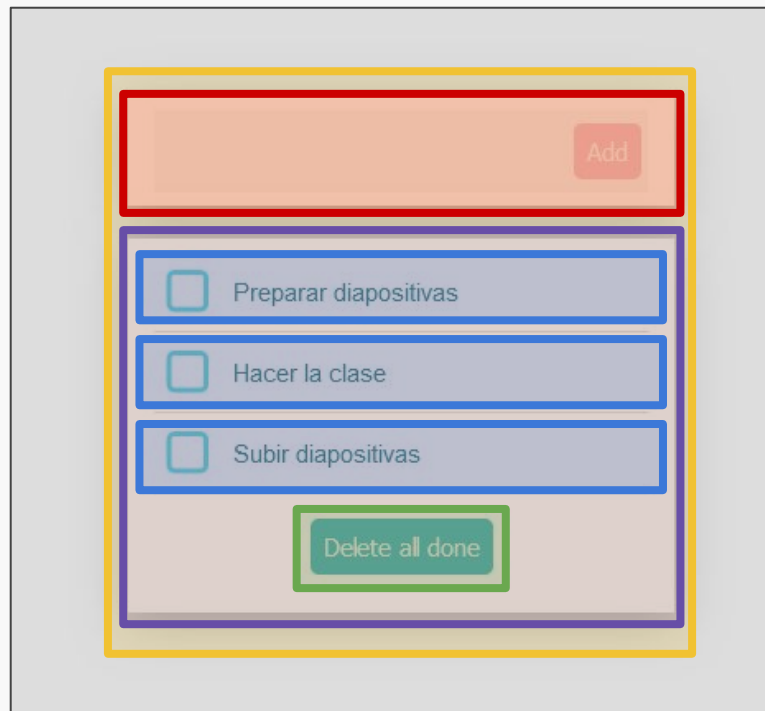


```
const Checkbox = props => {
  const {
    onChange,
    data: { id, description, done },
  } = props;
  return (
    <Fragment>
      <svg>...</svg>
      <label>
        <input name={id} type='checkbox' defaultChecked={done} />
        <svg>...</svg>
        <div>{description}</div>
      </label>
    </Fragment>
  );
};

const TaskList = props => {
  const { list, setList } = props;
  // ... callbacks
  return (
    <div className='todo-list'>
      {list.map(item => (
        <Checkbox key={item.id} data={item} />
      ))}
    </div>
  );
};
```

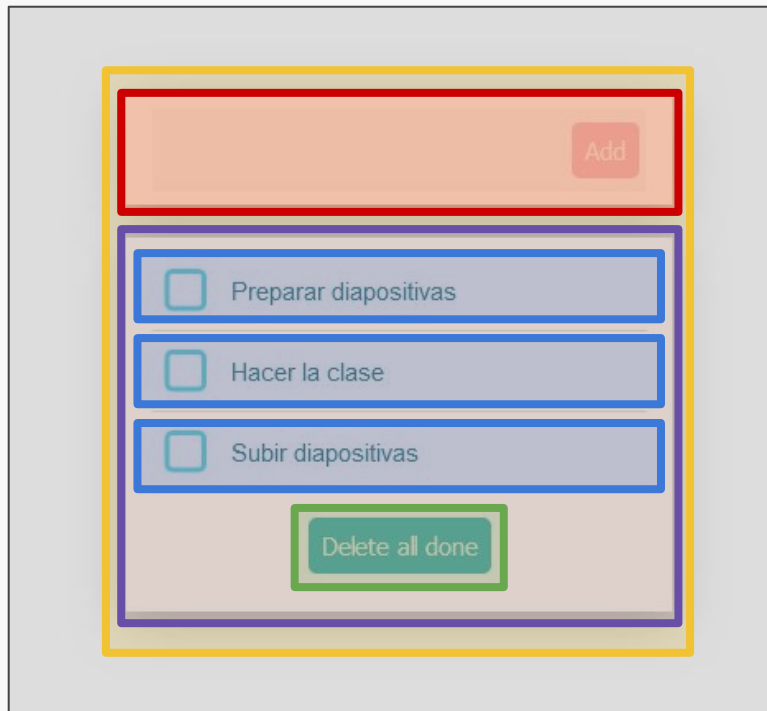

¿Cómo comenzar?

1. Identificar y nombrar componentes
2. Construir una versión estática
3. ... y los estados?



¿Cómo comenzar?

1. Identificar y nombrar componentes
2. Construir una versión estática
3. ... y los estados? ***Props y state***



Props: dando argumentos a los componentes

Props es la manera que tiene React para que los componentes puedan comunicarse entre sí, permitiendo el paso de información a descendientes. La sintaxis es similar a la de los atributos HTML, pero en lugar de solo *strings*, se puede pasar cualquier tipo de valor

```
function Avatar({ person, size }) {  
  return (  
    <img  
      className='avatar'  
      src={getImageUrl(person)}  
      alt={person.name}  
      width={size}  
      height={size}  
    />  
  );  
}
```

```
function Profile() {  
  return (  
    <Card>  
      <Avatar  
        size={100}  
        person={{  
          name: 'Katsuko Saruhashi',  
          imageId: 'Yfe0qp2',  
        }}  
      />  
    </Card>  
  );  
}
```

Props: dando argumentos a los componentes

Props es la manera que tiene React para que los componentes puedan comunicarse entre sí, permitiendo el paso de información a descendientes. La sintaxis es similar a la de los atributos HTML, pero en lugar de solo *strings*, se puede pasar cualquier tipo de valor

```
function Avatar({ person, size }) {  
  return (  
    <img  
      className='avatar'  
      src={getImageUrl(person)}  
      alt={person.name}  
      width={size}  
      height={size}  
    />  
  );  
}
```

```
function Profile() {  
  return (  
    <Card>  
      <Avatar  
        size={100}  
        person={{  
          name: 'Katsuko Saruhashi',  
          imageId: 'Yfe0qp2',  
        }}  
      />  
    </Card>  
  );  
}
```

Props: dando argumentos a los componentes

Props es la manera que tiene React para que los componentes puedan comunicarse entre sí, permitiendo el paso de información a descendientes. La sintaxis es similar a la de los atributos HTML, pero en lugar de solo *strings*, se puede pasar cualquier tipo de valor

```
function Avatar({ person, size }) {  
  return (  
    <img  
      className='avatar'  
      src={getImageUrl(person)}  
      alt={person.name}  
      width={size}  
      height={size}  
    />  
  );  
}
```

```
function Profile() {  
  return (  
    <Card>  
      <Avatar  
        size={100}  
        person={{  
          name: 'Katsuko Saruhashi',  
          imageId: 'Yfe0qp2',  
        }}  
      />  
    </Card>  
  );  
}
```

State: recordando valores al interactuar

Propiedad interna de un componente que puede cambiar en el tiempo. Requiere el uso del *hook* `useState`, que retorna un arreglo con dos valores:

1. El estado inicial del componente
2. Una función para setear un nuevo estado

```
const [todoList, setTodoList] = useState([]);
```



Valor actual



Función para actualizar *state*

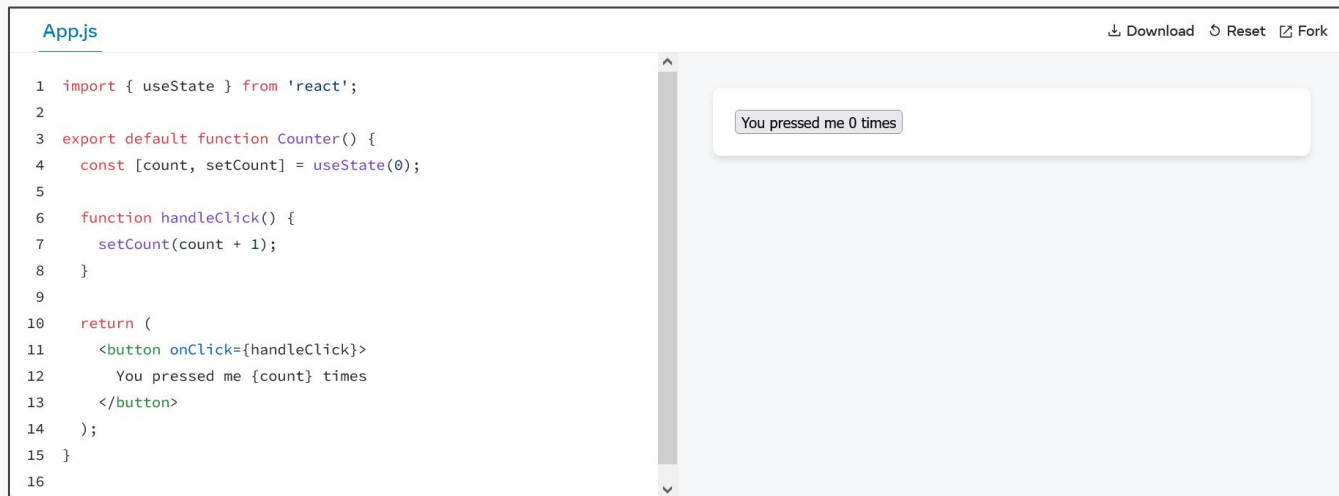


Valor inicial

State: recordando valores al interactuar

Propiedad interna de un componente que puede cambiar en el tiempo. Requiere el uso del *hook* `useState`, que retorna un arreglo con dos valores:

1. El estado inicial del componente
2. Una función para setear un nuevo estado

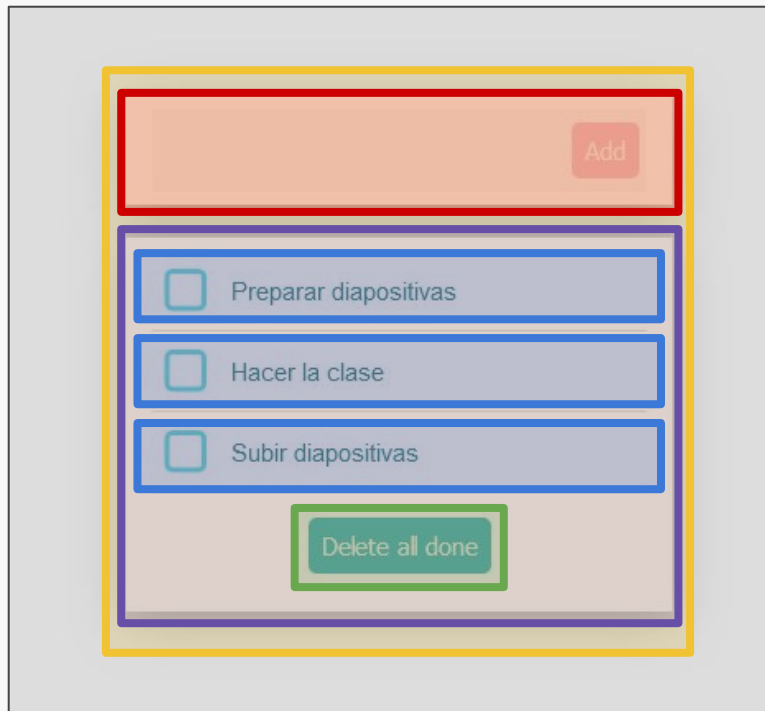


The screenshot shows a code editor with a file named `App.js`. The code defines a `Counter` component that uses `useState` to manage a `count` state. The initial state is 0. A `handleClick` function increments the count by 1. The component renders a button that, when clicked, updates the state and displays the current count. The right side of the editor shows the rendered output: a button labeled "You pressed me 0 times".

```
1 import { useState } from 'react';
2
3 export default function Counter() {
4   const [count, setCount] = useState(0);
5
6   function handleClick() {
7     setCount(count + 1);
8   }
9
10  return (
11    <button onClick={handleClick}>
12      You pressed me {count} times
13    </button>
14  );
15 }
16
```

¿Cómo comenzar?

1. Identificar y nombrar componentes
2. Construir una versión estática
3. Generación dinámica con *props*
4. Manejo de estado con *state*



¿Cómo comenzar?

Importante *#Cambios en props o state de cualquier componente harán que haga render nuevamente*

- 3. Generación dinámica con *props*
- 4. Manejo de estado con *state*



Virtual DOM

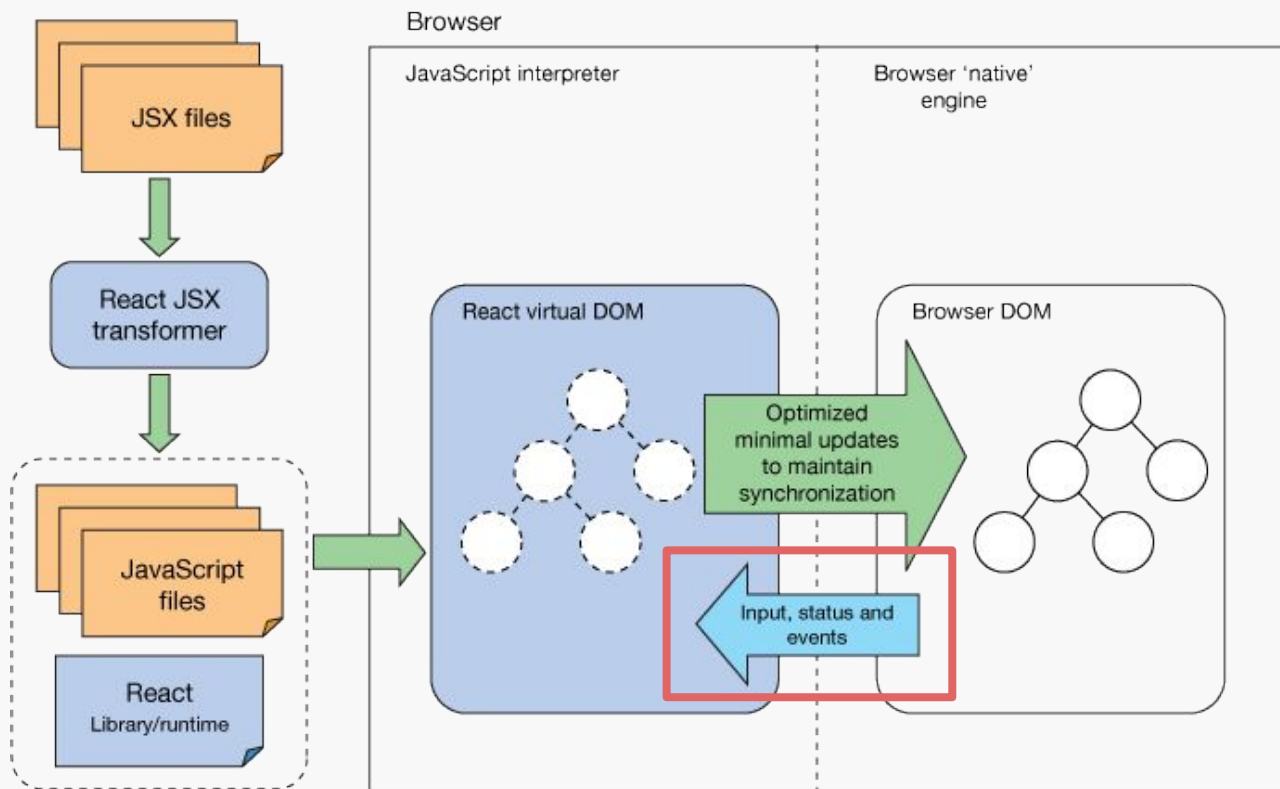


Figura: [React: Create maintainable, high-performance UI components](#) | IBM Developer

¿Cómo se ve una aplicación mínima en React?

1. Requiere importar algunos elementos
2. Definimos componentes (como función o clase)
3. Dibujamos los elementos con el método render

```
import React from 'react';  
import ReactDOM from 'react-dom/client';
```

1

```
function HelloMessage({ name }) {  
  return <div>Hello {name}</div>;  
}
```

2

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(<HelloMessage name='Antonio' />);
```

3

Agenda

✨ React

Conceptos importantes

Pensando en componentes

✨ Manejo de eventos

Un proyecto React

Event handling

Eventos en React se manejan de manera similar a eventos en el DOM (en particular, de manera similar a la definición de eventos *inline*). En el DOM (con HTML y JS):

```
<button onclick="handleClick()">Click Me</button>
```

```
function handleClick() {  
  alert('Button clicked!');  
}
```

Handlers como props

En React, recordando que las funciones son objetos

```
import React from 'react';

function ButtonExample() {
  const handleClick = () => {
    alert('Button clicked!');
  };

  return <button onClick={handleClick}>Click Me</button>;
}

export default ButtonExample;
```

Ciclo de vida de un componente

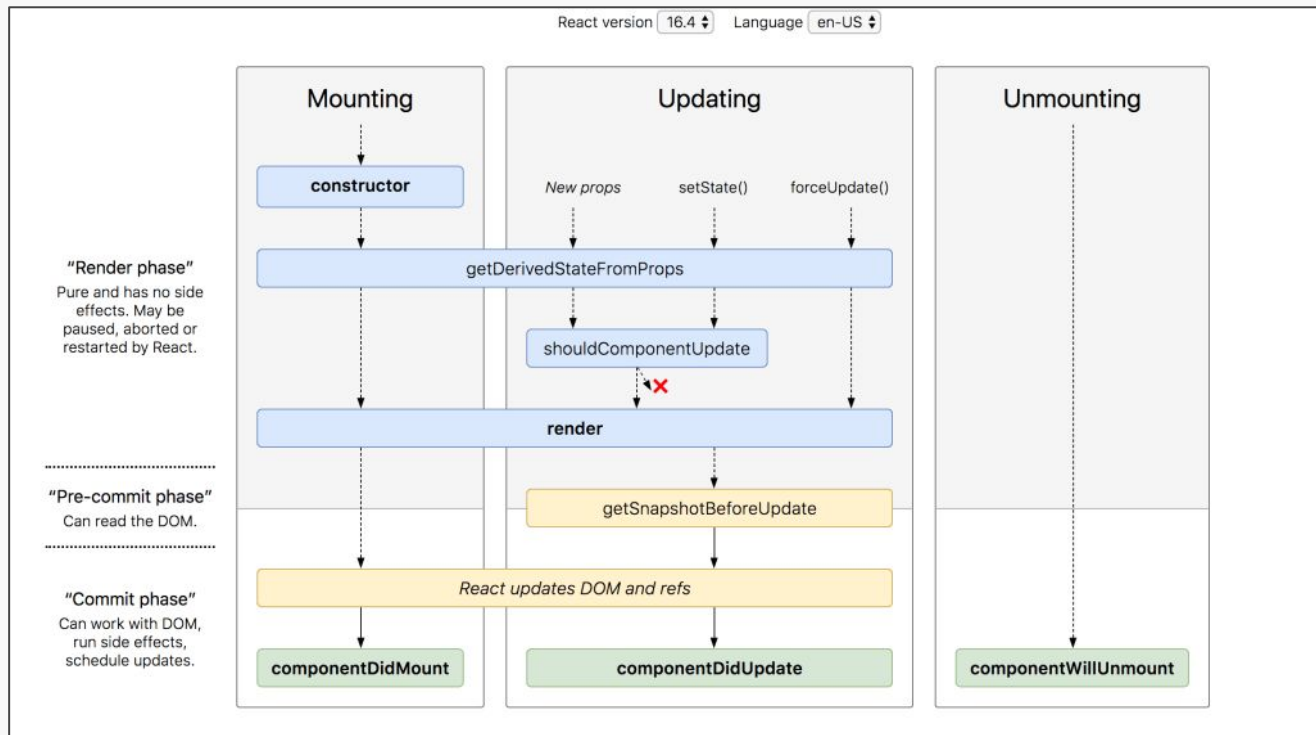


Figura: [React Lifecycle Methods. In our materials, there is a section... | by Zoe Bai | Medium](#) (Original pero ya no funciona)

Ciclo de vida de un componente

Inicialización (*Initialization*)

Fase en la que el componente va a empezar su trabajo. En esta fase se configuran el estado y las *props*. Esto se hace normalmente en el método constructor del componente

Montaje (*Mounting*)

En esta fase el componente React se crea e inserta en el DOM (se agrega como nodo en el DOM)

Actualización (*Updating*)

Fase en la que el estado del componente cambia y se produce el re-renderizado. Aquí, los datos del componente (estado y *props*) se actualizan en respuesta a eventos desde el usuario.

Desmontaje (*Unmounting*)

Última fase del ciclo de vida del componente. Como el nombre lo indica, el componente se desmonta del DOM

Hooks built-in

Hooks son funciones que permiten usar “estados” en componentes de React. Hacen posible utilizar React sin necesidad de definir clases, que antes eran necesarias para hacer seguimiento del estado de un componente y redibujarlo de ser necesario. **En resumen, los hooks son APIs que permiten utilizar estados (y otras características) en un componente funcional de React**

- State hooks (**useState**, useReducer)
- Context hooks (**useContext**)
- Ref hooks (useRef, useImperativeHandle)
- Effect hooks (**useEffect**)
- Performance hooks (useMemo, useCallback)
- Otros adicionales

useState - variables de estado

Este *hook* retorna un arreglo con dos valores:

1. El estado inicial del componente
2. Una función para setear un nuevo estado

```
import { useState } from 'react';

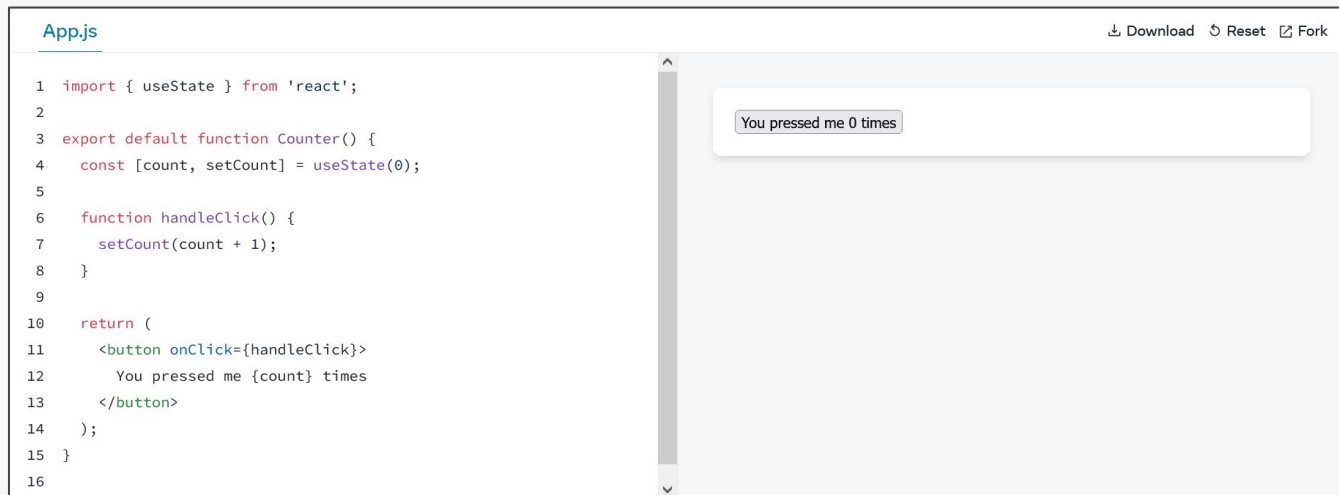
function MyComponent() {
  const [age, setAge] = useState(42);
  const [name, setName] = useState('Taylor');
  // ...
```

```
function handleClick() {
  setName('Robin');
}
```

useState - variables de estado

Este *hook* retorna un arreglo con dos valores:

1. El estado inicial del componente
2. Una función para setear un nuevo estado



The screenshot shows a code editor with a file named 'App.js'. The code defines a 'Counter' component that uses the 'useState' hook to manage a 'count' state, initialized to 0. A 'handleClick' function increments the count by 1. The component returns a button that displays the current count. To the right of the code editor, a preview of the rendered component is shown, displaying a button that says 'You pressed me 0 times'.

```
App.js
1 import { useState } from 'react';
2
3 export default function Counter() {
4   const [count, setCount] = useState(0);
5
6   function handleClick() {
7     setCount(count + 1);
8   }
9
10  return (
11    <button onClick={handleClick}>
12      You pressed me {count} times
13    </button>
14  );
15 }
16
```

Download Reset Fork

You pressed me 0 times

useContext - suscripción a un contexto

```
function MyPage() {  
  return (  
    <ThemeContext.Provider value='dark'>  
      <Form />  
    </ThemeContext.Provider>  
  );  
}
```

Declaración de contexto

El hook **useContext** permite suscribirse a un contexto, que puede estar mucho más arriba en el árbol, y acceder a su valor. Un contexto se declara a partir de un *context provider*, y su valor puede ser desde un simple string a un objeto con muchas *properties*

Suscripción a contexto

```
import { useContext } from 'react';  
  
function Button() {  
  const theme = useContext(ThemeContext);  
  // ...  
}
```

useEffect - sincronizar componentes

Al usar `useEffect` podemos sincronizar un componente con un sistema externo, permitiéndonos ejecutar “efectos secundarios” (no de UI) ante eventos en el ciclo de vida de un componentes. Se gatilla en cada renderizado: cuidado con ciclos infinitos

```
useEffect(() => console.log('mount'), []);  
useEffect(() => console.log('data1 update'), [data1]);  
useEffect(() => console.log('any update'));  
useEffect(() => () => console.log('data1 update or unmount'), [data1]);  
useEffect(() => () => console.log('unmount'), []);
```

useEffect - sincronizar componentes

Al usar `useEffect` podemos sincronizar un componente con un sistema externo, permitiéndonos ejecutar “efectos secundarios” (no de UI) ante eventos en el ciclo de vida de un componentes. Se gatilla en cada renderizado: cuidado con ciclos infinitos

```
import { useState, useEffect } from 'react';
import { createConnection } from './chat.js';

function ChatRoom({ roomId }) {
  const [serverUrl, setServerUrl] = useState('https://localhost:1234');

  useEffect(() => {
    const connection = createConnection(serverUrl, roomId);
    connection.connect();
    return () => {
      connection.disconnect();
    };
  }, [serverUrl, roomId]);
  // ...
}
```

Agenda

✨ React

Conceptos importantes

Pensando en componentes

Manejo de eventos

✨ Un proyecto React

Package managers: módulos en JavaScript

Así como Python tiene pip y Ruby tiene gemas...



Vite

Vite (/vit/) es una herramienta de *front-end* que facilita el desarrollo y construcción de aplicaciones, y que puede trabajar con distintas librerías (Vue, React, Preact, Svelte o Lit)

¿Qué ofrece?

- Fácil iniciar un servidor (vite)
- Ofrece hot-reloading (“HMR”)
- Soporta TypeScript, JSX, CSS y, por supuesto, JavaScript
- Toma muchas decisiones por uno, buscando optimizar



Vite (docs): Herramienta para iniciar proyectos rápidamente



Vite



Search

Ctrl K

Guide

Config

Plugins

Resources ▾

Version ▾

⌘A ▾



ViteConf 2025

The Build Tool for the Web

Vite is a blazing fast frontend build tool powering
the next generation of web applications.

Get started



GitHub

¡Vamos al código!

1. Creemos un proyecto con Vite
2. exploremos los archivos que vienen en un proyecto de Vite
3. Veamos qué ocurre dentro del navegador con la aplicación

Aplicación de ejemplo creada con Vite:

github.com/IIC2513/ToDoApp

Agenda

✨ React

Conceptos importantes

Pensando en componentes

Manejo de eventos

Un proyecto React

✨ ¿Qué se viene pronto?

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 3 (17/3 - 23/3)		JavaScript pt. 1 (Intro)		JavaScript pt. 2 (DOM y async)	Ayudantía: JavaScript		
Semana 4 (24/3 - 30/3)		HOY		Exploración y consumo de APIs	Ayudantía: React		
Semana 5 (31/3 - 06/4)		Node.js		Desarrollando servidores	Ayudantía: Consumo de APIs		
Semana 6 (07/4 - 13/4)		Construcción de APIs		QA, CI & CD	Ayudantía: Construcción de APIs		
Semana 7 (14/4 - 20/4)		Fundamentos de front-end		Jueves Santo	Viernes Santo	Sábado Santo	Domingo de Resurrección

Próximas fechas importantes: evaluaciones

	Evaluaciones						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 3 (17/3 - 23/3)							
Semana 4 (24/3 - 30/3)	Control 1	HOY			Inicio T1 React		
Semana 5 (31/3 - 06/4)					Fin T1 (React)		
Semana 6 (07/4 - 13/4)	Control 2				Inicio T2 API		
Semana 7 (14/4 - 20/4)				Jueves Santo	Viernes Santo	Sábado Santo	Domingo de Resurrección

Clase 5

React

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl